

# CYBER LAKSHYA

## Database & Data Engineering Research and Implementation Report

Smart India Hackathon 2026

Team: Cyber Lakshya

Role: Het Pansara — Database & Data Engineering

### 1. Executive Summary

Cyber Lakshya proposes a centralized cybersecurity and Data Center Infrastructure Management (DCIM) portal for AICTE. The database and data-engineering layer is the central source of truth for infrastructure assets, software and licenses, monitoring telemetry, alerts, audit information, and relationships between these resources.

The proposed database approach is PostgreSQL-based. The design uses relational modeling for core entities, appropriate constraints and foreign keys for integrity, indexes for frequent queries, JSONB where device-specific configuration is semi-structured, and partitioning for very large telemetry/event tables when the workload justifies it.

The implementation will be incremental: requirements → entity identification → ER model → relational schema → PostgreSQL prototype → asset inventory → software/license management → monitoring storage → performance optimization → security/audit → Go API integration → AI/ML and dashboard integration.

### 2. Problem Context

The SIH problem statement describes fragmented infrastructure management, manual processes, limited visibility, software-license compliance challenges, and difficult user-access management. The proposed portal needs centralized management of servers, firewalls, network devices, load balancers, licenses, users, hardware assets, monitoring, reporting, automation, and security.

From the database perspective, the key challenge is to create a reliable source of truth that can represent both relatively stable inventory information and high-volume operational telemetry.

| Problem                   | Database/Data Engineering Response                                                    |
|---------------------------|---------------------------------------------------------------------------------------|
| Fragmented infrastructure | Centralized PostgreSQL data model and common asset identity.                          |
| Manual inventory          | Structured asset records with lifecycle/status fields and API-based updates.          |
| Limited visibility        | Unified asset, monitoring, alert and reporting data.                                  |
| License compliance        | Software, license, allocation, expiry and usage relationships.                        |
| Hardware lifecycle        | Warranty, maintenance, location and status tracking.                                  |
| Large telemetry volume    | Indexes, time-based partitioning where justified, retention and aggregation strategy. |
| Auditability              | Immutable-style application audit records with actor, action, resource and timestamp. |

### 3. My Assigned Role

Het Pansara — Database & Data Engineering. The responsibility is to architect the PostgreSQL storage layer and the data structures needed by the backend, dashboard, monitoring system, and AI/ML components.

| Responsibility              | Expected Outcome                                                            |
|-----------------------------|-----------------------------------------------------------------------------|
| PostgreSQL architecture     | Scalable, consistent and secure relational data layer.                      |
| Software license repository | Track software, licenses, allocations, usage, compliance and renewal dates. |
| Hardware inventory          | Track servers, switches, firewalls, load balancers and other assets.        |
| Monitoring data design      | Store infrastructure metrics and support efficient retrieval.               |
| AI/ML data support          | Provide clean, timestamped features for anomaly detection.                  |
| Integration support         | Expose database structures and query requirements to the Go backend.        |

## 4. Research Findings

### 4.1 Centralized Asset Inventory

NIST SP 800-53 CM-8 requires an inventory of system components that accurately reflects the system, includes relevant components, avoids duplicate accounting, has an appropriate level of granularity, and is reviewed and updated. NIST also describes centralized inventories as an implementation option. This strongly supports the project's centralized asset model.

CISA's Cybersecurity Performance Goals similarly recommend maintaining a regularly updated inventory of organizational assets with IP information. These references support treating inventory as a security capability, not merely an administrative list.

#### Design implication

Every infrastructure component should have a unique asset identity and lifecycle status. Shared attributes belong in a common asset model; device-specific attributes can be represented in specialized tables or carefully controlled JSONB.

## 4.2 Software Asset and License Management

NIST describes Software Asset Management as a component of continuous monitoring and highlights integrated software inventory information for security and operational decision-making. The project therefore needs to distinguish the software product itself from a purchased license and from an individual license allocation/installation.

| Entity             | Purpose                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------|
| Software           | Product identity, vendor, version and category.                                           |
| License            | Entitlement, quantity, type, purchase and expiry information.                             |
| License Assignment | Maps a license to a server/user and records allocation lifecycle.                         |
| Vendor             | Stores supplier/manufacturer information.                                                 |
| Compliance status  | Derived or stored state indicating valid, expiring, expired or over-allocated conditions. |

## 4.3 PostgreSQL as the Core Database

PostgreSQL is suitable because the core domain contains strongly related entities requiring referential integrity, constraints, joins and transactional updates. Current PostgreSQL documentation also provides capabilities relevant to this project, including indexing, JSONB, partitioning, concurrency control, performance tools and replication.

## 4.4 JSONB for Variable Device Configuration

Different infrastructure devices can expose different configuration fields. PostgreSQL JSONB can store appropriate semi-structured configuration while core identifiers and relationships remain relational. PostgreSQL documents JSONB indexing through GIN indexes.

## 4.5 Indexing and Query Performance

Indexes should be designed around actual access patterns rather than added to every column. Candidate indexed fields include asset ID, hostname, IP address, serial number, status, location ID, license ID and timestamp fields used for monitoring queries.

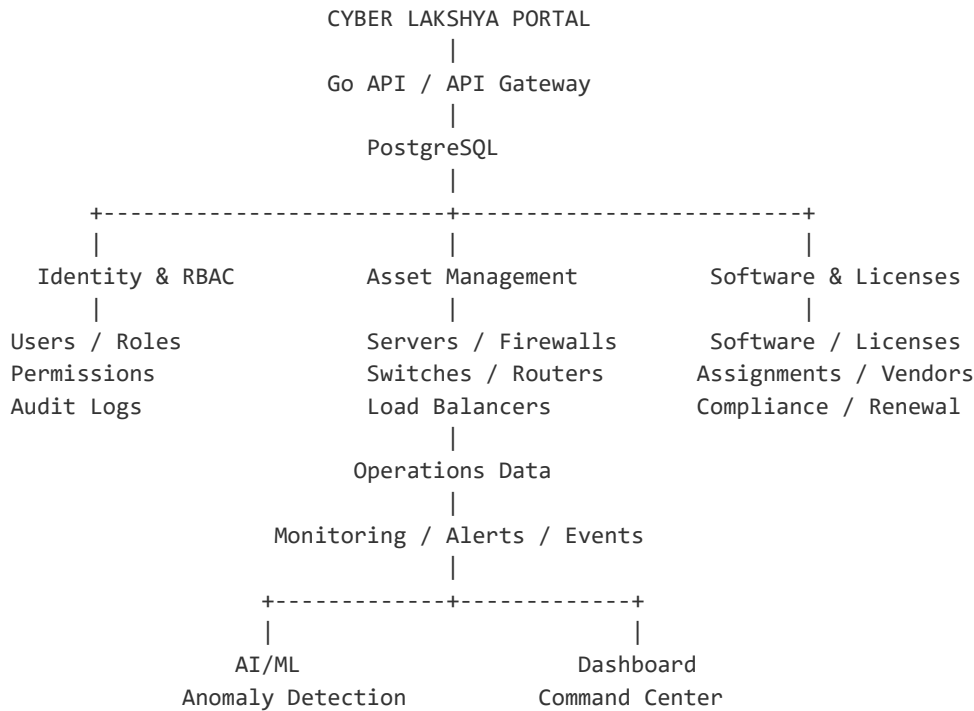
## 4.6 Partitioning for High-Volume Telemetry

PostgreSQL supports declarative partitioning. Partitioning can improve performance in suitable workloads by allowing queries to prune irrelevant partitions. The project's likely candidates are monitoring metrics, network events, security events and potentially audit data if volumes become sufficiently large. Partitioning will be applied based on measured workload rather than as a blanket rule.

## 4.7 Blockchain Relevance

Because the SIH domain is Blockchain & Cybersecurity, blockchain should not replace PostgreSQL. A practical architecture is to keep the operational source of truth in PostgreSQL and optionally anchor selected high-value audit/provenance records to a tamper-evident blockchain or permissioned ledger. NIST's 2026 draft BloSS@M work is a useful research reference because it discusses blockchain-based secure software asset management and immutable lifecycle tracking.

## 5. Proposed Data Architecture



The database acts as the central data layer. The Go backend is responsible for API-level business logic and authorization integration. The frontend consumes API responses rather than accessing PostgreSQL directly.

## 6. Proposed Core Entities

| Entity/Table        | Main Purpose                                                 | Priority |
|---------------------|--------------------------------------------------------------|----------|
| users               | Portal users and identity references.                        | High     |
| roles / permissions | RBAC relationships and authorization metadata.               | High     |
| assets              | Common identity and lifecycle for infrastructure components. | Critical |
| servers             | Server-specific technical information.                       | Critical |
| network_devices     | Firewall, switch, router and related device data.            | Critical |
| locations           | Data center/site/rack/location hierarchy.                    | High     |
| vendors             | Manufacturer and supplier references.                        | Medium   |
| software            | Software product catalog.                                    | Critical |
| licenses            | License entitlement and lifecycle.                           | Critical |
| license_assignments | License-to-asset/user allocation history.                    | Critical |
| warranties          | Warranty periods and lifecycle.                              | High     |
| maintenance_records | Maintenance schedules and records.                           | Medium   |
| monitoring_metrics  | Timestamped infrastructure telemetry.                        | Critical |
| alerts              | Detected operational/security events.                        | High     |
| audit_logs          | Record of administrative/data changes.                       | Critical |

## 7. Proposed Relationships

```

USER ----< USER_ROLE >---- ROLE
ASSET ----< SERVER
ASSET ----< NETWORK_DEVICE
ASSET ----< WARRANTY
ASSET ----< MAINTENANCE_RECORD
ASSET ----< MONITORING_METRIC
ASSET ----< ALERT

```

```

LOCATION ----< ASSET
VENDOR ----< SOFTWARE
SOFTWARE ----< LICENSE
LICENSE ----< LICENSE_ASSIGNMENT
ASSET ----< LICENSE_ASSIGNMENT

```

```

USER ----< AUDIT_LOG
ASSET ----< AUDIT_LOG

```

The final schema will be refined after coordination with the backend, RBAC, monitoring and frontend members. The goal is to avoid both under-modeling and unnecessary table complexity.

## 8. Implementation Plan

1. Confirm functional requirements and the exact fields required by the backend, dashboard and AI/ML model.
2. Finalize the entity inventory and produce the ER diagram.
3. Convert the ER model into a normalized PostgreSQL schema with primary keys, foreign keys, unique constraints and validation rules.
4. Implement the core asset, software, license, location and vendor tables.
5. Implement license allocation, expiry and compliance logic.
6. Implement hardware inventory and lifecycle information including warranty and maintenance.
7. Implement monitoring and alert storage with appropriate indexes and a timestamp strategy.
8. Benchmark representative queries and insert workloads; add indexes based on measured access patterns.
9. Introduce time-based partitioning for telemetry/event tables if the expected data volume and query pattern justify it.
10. Add database security controls, least-privilege roles, backup/recovery procedures and audit logging.
11. Integrate the database with Jiya's Go backend and expose the required API operations.
12. Provide ML-ready monitoring datasets to Chaitanya and dashboard-ready summaries to Riddhi.
13. Create a final demo dataset and test the complete workflow end-to-end.

## 9. Phased Roadmap

| Phase   | Work                    | Deliverable                                      |
|---------|-------------------------|--------------------------------------------------|
| Phase 1 | Requirements & research | Entity list + research notes                     |
| Phase 2 | Data modeling           | ER diagram + schema specification                |
| Phase 3 | PostgreSQL foundation   | Database + core tables + constraints             |
| Phase 4 | Asset & license modules | Inventory + license lifecycle                    |
| Phase 5 | Monitoring              | Metrics + alerts + query strategy                |
| Phase 6 | Scalability             | Indexes + partitioning/retention where justified |
| Phase 7 | Security                | Least privilege + audit + backup plan            |
| Phase 8 | Integration             | Go API + AI/ML + dashboard                       |
| Phase 9 | Testing & demo          | End-to-end working prototype                     |

## 10. Example End-to-End Workflows

### 10.1 Hardware Registration

#### New Server

- > Create asset record
- > Link location
- > Add server-specific configuration
- > Record warranty
- > Mark lifecycle status = ACTIVE
- > Begin monitoring
- > Dashboard reflects the new asset

### 10.2 License Compliance

#### Software

- > License purchased
- > License quantity recorded
- > License assigned to server/user
- > Usage count calculated
- > Expiry checked
- > Compliance status generated
- > Dashboard shows compliant / expiring / expired / over-allocated

### 10.3 Anomaly Detection

#### Server telemetry

- > monitoring\_metrics
- > API/data pipeline
- > Isolation Forest
- > anomaly result
- > alerts
- > dashboard notification

## 11. Database Performance Strategy

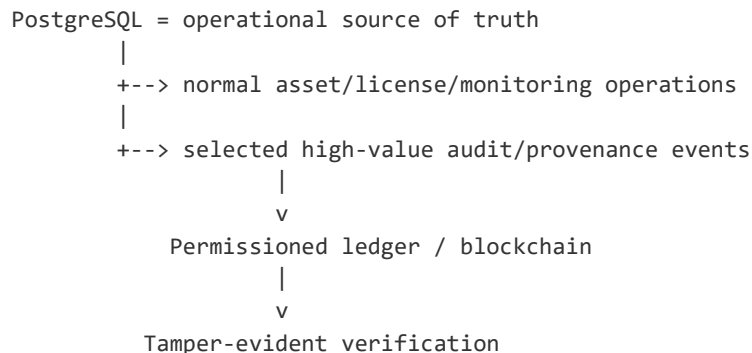
| Technique             | Why It Matters                                            | Planned Use                                        |
|-----------------------|-----------------------------------------------------------|----------------------------------------------------|
| Indexes               | Faster lookups and filtering for common queries.          | Asset, status, IP, license and time-based queries. |
| Partitioning          | Limits scans to relevant portions of very large tables.   | Telemetry/events when volume justifies it.         |
| Connection pooling    | Avoids opening a new DB connection for every API request. | Go backend integration.                            |
| Batch inserts         | Reduces per-row overhead during telemetry ingestion.      | Monitoring/event ingestion where appropriate.      |
| Retention/aggregation | Controls long-term storage growth.                        | Raw telemetry vs summarized metrics.               |
| Query analysis        | Prevents assumptions about performance.                   | Benchmark with EXPLAIN/EXPLAIN ANALYZE.            |
| Replication/backup    | Improves resilience and recovery.                         | Design target for later prototype maturity.        |

## 12. Security and Data Governance Plan

- Use separate database roles for application access and administrative operations.
- Apply least privilege; the application account should receive only the permissions required by the API.
- Protect database connections and credentials using secure configuration/secrets management.
- Do not store sensitive secrets such as raw passwords or private keys in ordinary asset tables.
- Maintain audit records for important administrative and data-changing actions.
- Use constraints and foreign keys to prevent invalid references and orphaned records.
- Back up the database and define a tested recovery procedure.
- Define retention and access rules for monitoring, audit and security-event data.
- Coordinate with the cybersecurity lead so database controls align with the project's Zero Trust and RBAC design.

## 13. Blockchain Integration — Recommended Scope

The project domain includes Blockchain & Cybersecurity, but using blockchain for every database operation would add unnecessary complexity and latency. The recommended prototype architecture is hybrid.



Potential blockchain records include license acquisition/transfer events, critical asset lifecycle changes, or selected administrative audit events. The exact event set should be finalized after discussing the blockchain component with the cybersecurity team.

## 14. Team Integration Plan

| Team Member          | Dependency on Het's Data Layer                                                       |
|----------------------|--------------------------------------------------------------------------------------|
| Maharshi Trivedi     | Security architecture, audit requirements and security-state data.                   |
| Siddharthsinh Raulji | Firewall/network-device data requirements and compliance-related fields.             |
| Chaitanya Thakar     | RBAC metadata and ML telemetry/features for anomaly detection.                       |
| Jiya Bhayani         | Go API integration, CRUD operations, queries and connection pooling.                 |
| Riddhi Odedra        | Dashboard data, asset summaries, license metrics, alerts and flow-map relationships. |

### Immediate coordination

Before finalizing the schema, Het should conduct a short data-contract discussion with Jiya and Chaitanya: which API objects are required, which telemetry fields feed Isolation Forest, and which dashboard queries must be fast.

## 15. Testing Plan

- Functional tests: create, read, update, retire and search assets.
- License tests: allocation, release, expiry, over-allocation and renewal scenarios.
- Integrity tests: foreign keys, unique serial numbers, invalid references and required fields.
- Performance tests: large asset queries and bulk telemetry inserts.
- Scalability tests: progressively increase mock assets and monitoring records.
- Security tests: verify database roles cannot perform unauthorized operations.
- Recovery tests: restore a backup into a test environment.
- Integration tests: Go API → PostgreSQL → frontend and ML consumers.

## 16. Expected Final Deliverables for Het

- Database architecture diagram.
- ER diagram.
- PostgreSQL schema specification.
- Working PostgreSQL database with seed/demo data.
- Hardware asset inventory tables and operations.
- Software/license management tables and compliance queries.
- Monitoring/telemetry storage design.
- Indexing strategy and performance notes.
- Partitioning/retention strategy for high-volume data.
- Audit-log structure.
- Database integration contract for the Go backend.
- ML data contract for anomaly detection.
- Dashboard query/data contract.
- Testing evidence and demo scenarios.



## 17. Risks and Mitigations

| Risk                           | Impact                              | Mitigation                                                                            |
|--------------------------------|-------------------------------------|---------------------------------------------------------------------------------------|
| Overly complex schema          | Slower development and integration. | Start with core entities; add tables only when requirements justify them.             |
| Poor query performance         | Dashboard/monitoring becomes slow.  | Use query-driven indexing and benchmark with realistic data.                          |
| Telemetry volume grows rapidly | Storage and query costs increase.   | Partition, retain, aggregate and archive according to requirements.                   |
| Inconsistent asset identities  | Duplicate or unreliable inventory.  | Stable asset IDs, unique constraints and controlled lifecycle updates.                |
| Schema changes block teammates | Integration delays.                 | Agree API/data contracts before major changes.                                        |
| Sensitive data exposure        | Security/compliance risk.           | Least privilege, encryption, secret management and audit logging.                     |
| Blockchain over-engineering    | Unnecessary complexity.             | Use blockchain selectively for tamper-evident provenance rather than primary storage. |

## 18. Research References

- **NIST SP 800-53 Rev. 5.1 — CM-8 System Component Inventory**  
[https://csrc.nist.gov/CSRC/media/Projects/risk-management/800-53%20Downloads/800-53r5/SP\\_800-53\\_v5\\_1-derived-OSCAL.pdf](https://csrc.nist.gov/CSRC/media/Projects/risk-management/800-53%20Downloads/800-53r5/SP_800-53_v5_1-derived-OSCAL.pdf)
- **NIST Software Asset Management: Continuous Monitoring**  
<https://csrc.nist.gov/pubs/pd/2015/09/16/software-asset-management-continuous-monitoring/final>
- **NIST IR 8500A — Blockchain-Based Secure Software Assets Management (BloSS@M)**  
<https://csrc.nist.gov/pubs/ir/8500/a/ipd>
- **CISA Cross-Sector Cybersecurity Performance Goals**  
<https://www.cisa.gov/cross-sector-cybersecurity-performance-goals>
- **PostgreSQL Documentation**  
<https://www.postgresql.org/docs/current/>
- **PostgreSQL JSON Types / JSONB**  
<https://www.postgresql.org/docs/current/datatype-json.html>
- **PostgreSQL Table Partitioning**  
<https://www.postgresql.org/docs/current/ddl-partitioning.html>

## 19. Conclusion

Het Pansara's contribution is the data foundation of Cyber Lakshya. The immediate objective is not to build an unnecessarily large database, but to build a clean, secure and scalable source of truth that demonstrates the core SIH requirements. The implementation will prioritize centralized asset visibility, software-license lifecycle/compliance, monitoring data, auditability and integration with the Go backend, AI/ML pipeline and administrator dashboard.

The recommended next action is to finalize the entity list and ER diagram with the team. Once that is agreed, the PostgreSQL schema can be implemented and tested against realistic demo data before integration.

**Research note:** Standards and technical claims in this report were checked against current NIST, CISA and PostgreSQL documentation available in August 2026.